

Relational Model

INT191 Relational Database Concepts and Design

Dr. Sunisa Sathapornvajana

What we will learn

- * Relational Model Concepts and Terminology
- * Relational Key
- * Integrity Constraint

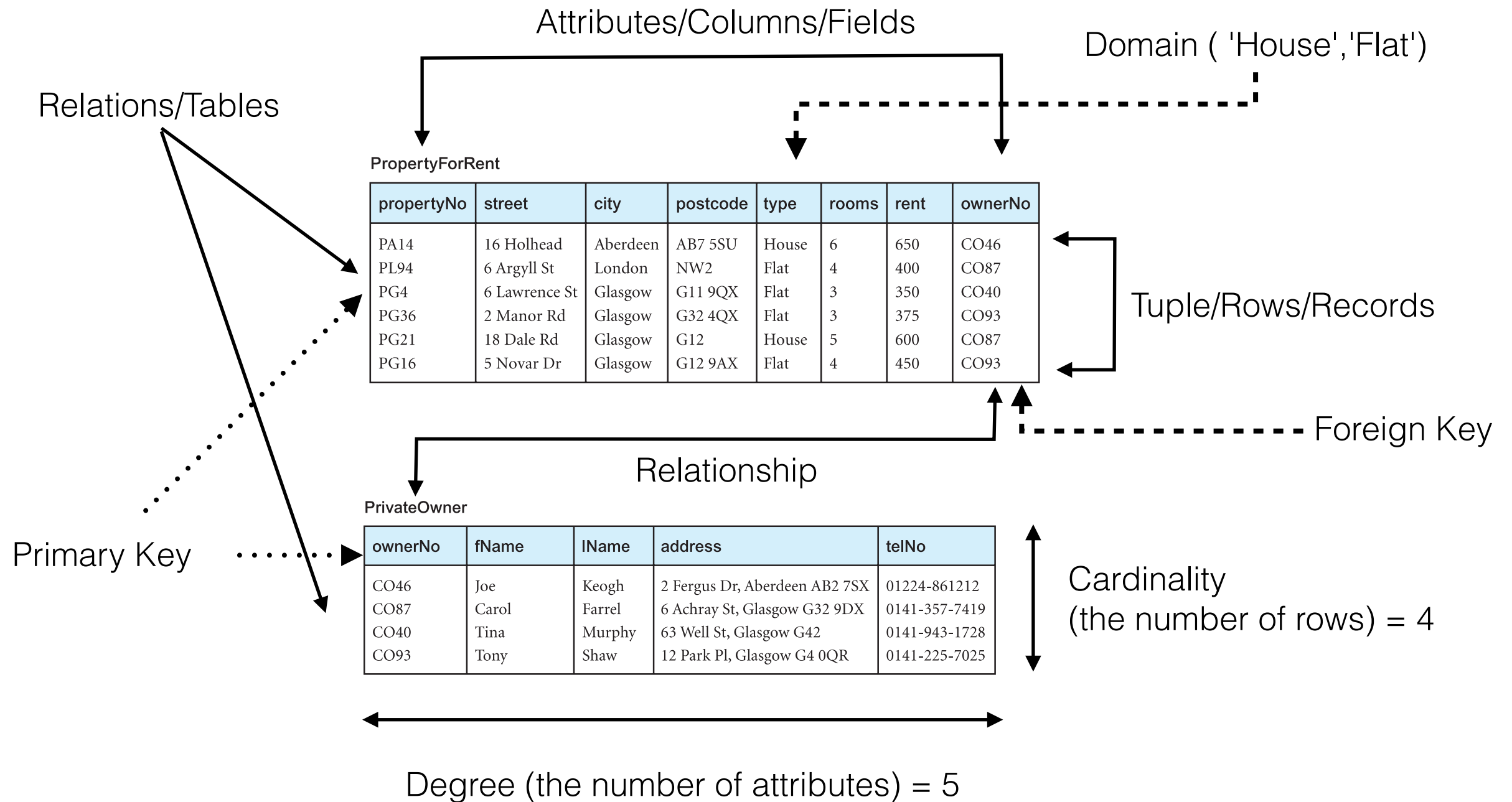
Relational Model Concepts

- * A Relation is a mathematical concept based on the ideas of sets
- * The model was first proposed by **Dr. E.F. Codd** of IBM Research in 1970 in the following paper:
 - * **"A Relational Model for Large Shared Data Banks,"
Communications of the ACM, June 1970**
- * The above paper caused a major revolution in the field of database management and earned Dr. Codd the coveted ACM Turing Award

The Relational Model

- * The relational model
 - * All data is logically structured within **relations** (**tables**).
 - * Each relation has a name and is made up of named **attributes** (**columns**) of data.
 - * Each **tuple** (**row**) contains one value per attribute.
- * A great strength of the relational model is this simple logical structure.
- * The relational model allows a high degree of data independence.
 - * Application programs must not be affected by modifications of the internal data representation, file organizations, etc.
- * A relational database (RDBMS) is a collection of normalized relations with distinct relation names.

Relational Model Terminology



Relational Model Terminology

*Relation

- * A relation is used to hold information about the objects to be represented in the database.
- * A relation is represented as a two-dimensional table, row and column.

*Attribute

- * Attribute is a column of a relation.
- * Attributes can appear in any order and the relation will still be the same relation.

*Tuple

- * A tuple is a row of a relation.
- * Tuples can appear in any order and the relation will still be the same relation.

Relational Model Terminology

*Cardinality

- * The cardinality of a relation is the number of tuples it contains.
- * The cardinality is determined from the particular instance of the relation at any given moment.

*Domain

- * A domain is the set of allowable values for an attribute.
- * Every attribute in a relation is defined on a domain.

Attribute	Domain Name	Meaning	Domain Definition
branchNo	BranchNumbers	The set of all possible branch numbers	character: size 4, range B001–B999
street	StreetNames	The set of all street names in Britain	character: size 25
city	CityNames	The set of all city names in Britain	character: size 15
postcode	Postcodes	The set of all postcodes in Britain	character: size 8
sex	Sex	The sex of a person	character: size 1, value M or F
DOB	DatesOfBirth	Possible values of staff birth dates	date, range from 1-Jan-20, format dd-mmm-yy
salary	Salaries	Possible values of staff salaries	monetary: 7 digits, range 6000.00–40000.00

Relational Model Terminology

*Schema

- * The Schema (or description) of a Relation [or a Database]:

 - * Denoted by $R(A_1, A_2, \dots, A_n)$.

 - * R is the name of the relation.

 - * The attributes of the relation are $A_1, A_2, \dots, A_n$.

- * Example:

STUDENTS (StudentNo, Firstname, Lastname, DOB, Email)

 - * STUDENTS is the relation name.

 - * Defined over the four attributes: StudentNo, Firstname, Lastname, DOB, Email

Relational Model Terminology

Informal Terms	Formal Terms
Table	
Column Header	
All possible column values	
Row	
Table Definition	

Properties of Relations

- *The relation has a name that is distinct from all other relation names in the relational schema.
- *Each **cell** of the relation contains exactly **one atomic (single) value**.
- *Each attribute has a distinct name.
- *The values of an attribute are all from the same domain.
- *Each tuple is distinct; there are no duplicate tuples.

Properties of Relations

- *The order of tuples has no significance.
- *The order of attributes has significance.
 - * We will consider the attributes in $R(A_1, A_2, \dots, A_n)$ and the values in $t = \langle v_1, v_2, \dots, v_n \rangle$ to be ordered.

* Example:

INSERT INTO students

VALUES (678979000, 'BOB', 'Kinsey', '2009-08-15', NULL);

Properties of Relations

- *Values in a tuple:

- * All values are considered **atomic**.

- * Each value in a tuple must be from the domain of the attribute for that column.

- * A special **null** value is used to represent values that are **unknown or not available or inapplicable** in certain tuples.

Integrity constraint

- * Integrity constraints are **a set of rules used to maintain the quality of information.**
- * The data **insertion, updating, deleting** and other processes must follow the rules.
- * Integrity constraint is used to guard against accidental damage to the database.

DML Operations on Relations

Syntax

* INSERT a tuple.

```
INSERT INTO table_name [(column_list)]  
VALUES (value1, value2,..., value3);
```

* UPDATE a tuple.

```
UPDATE table_name  
SET column = value [,column2 = value2,...]  
[WHERE condition(s)];
```

* DELETE a tuple.

```
DELETE table_name  
[WHERE condition(s)];
```

Integrity constraint

*Key constraints

- *Relational key is a set of attributes that **uniquely identifies each tuple in a relation.**

*Entity integrity Constraint

- *Every relation must have a primary key and it can not be null

*Referential integrity Constraint

- *The foreign key value must match **a primary key** value of some tuple in its home relation or
- *The foreign key value can be wholly **null**.

Relational Key

- * Relational key is a set of attributes that **uniquely identifies each tuple in a relation.**
- * Types of key
 - * Superkey
 - * Candidate key
 - * Primary key

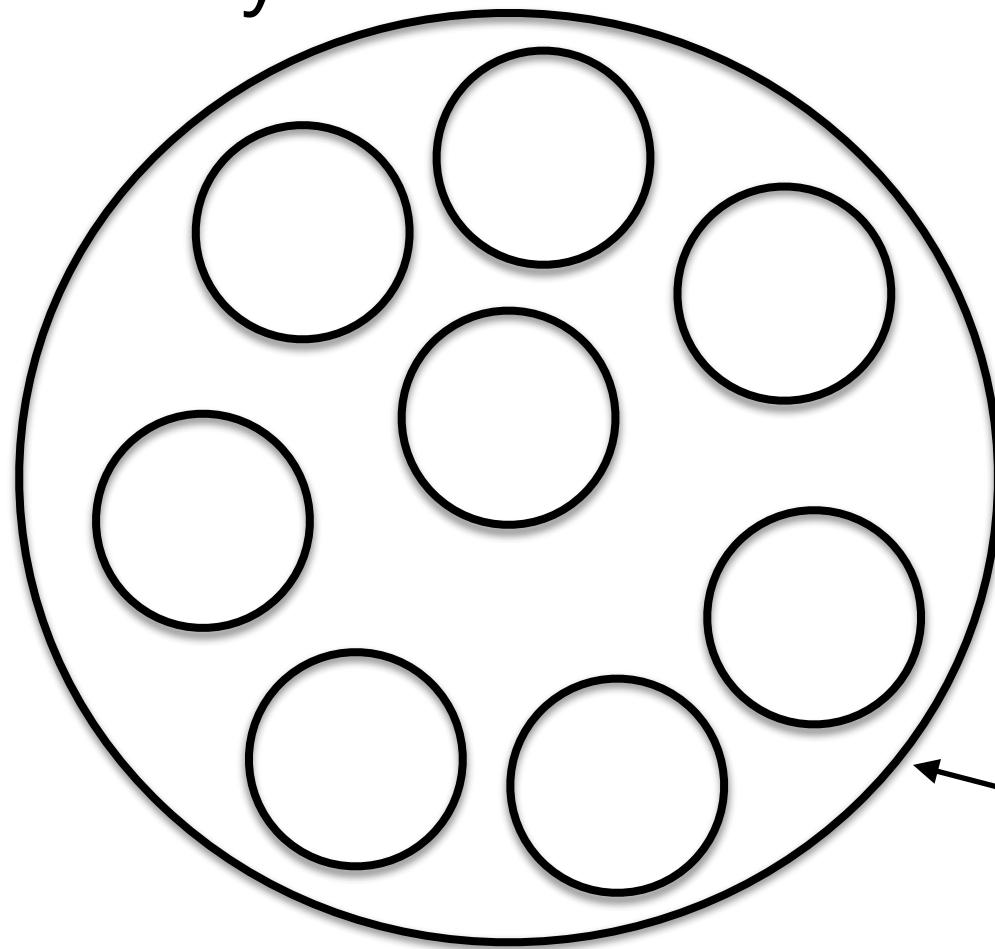
Superkey

- * A **superkey** is an attribute, or set of attributes, that uniquely identifies a tuple within a relation.
- * A superkey may contain additional attributes that are not necessary for unique identification.

Relational Keys

Students (StudentNo, Firstname, Lastname, DOB, Email)

SuperKey



← A Relation

Candidate key

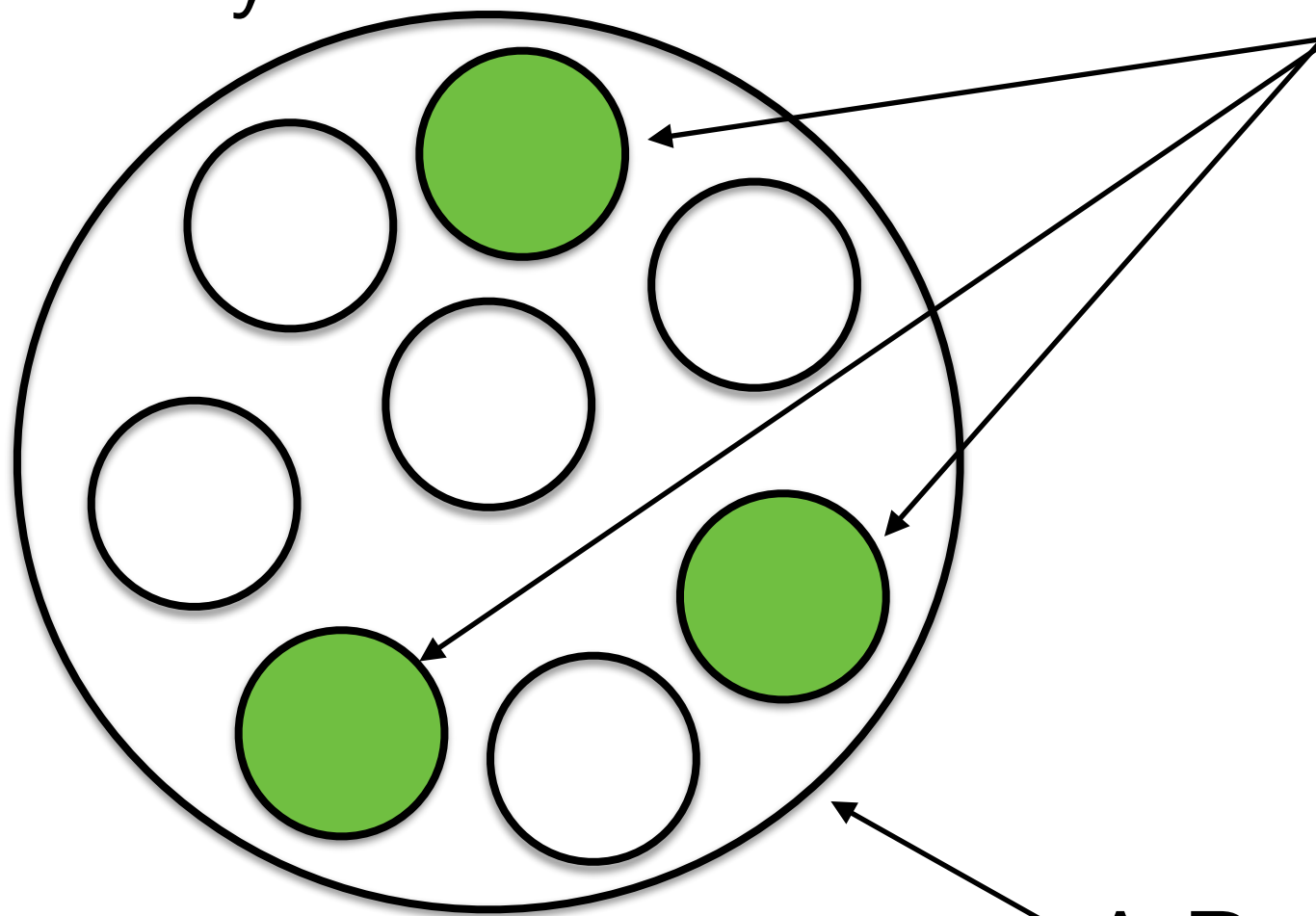
- * **Candidate key** is a superkey that no proper subset is a superkey within the relation.
- * A candidate key K for a relation R has two properties:
 - * **Uniqueness**. In each tuple of R , the values of K uniquely identify that tuple.
 - * **Irreducibility**. No proper subset of K has the uniqueness property. A "minimal" superkey
- * There may be several candidate keys for a relation.
- * When a key consists of more than one attribute, we call it a composite key.
- * Identifying a candidate key requires that we know the "real-world" meaning.

Relational Key

Students (StudentNo, Firstname, Lastname, DOB, Email)

SuperKey

Candidate Key



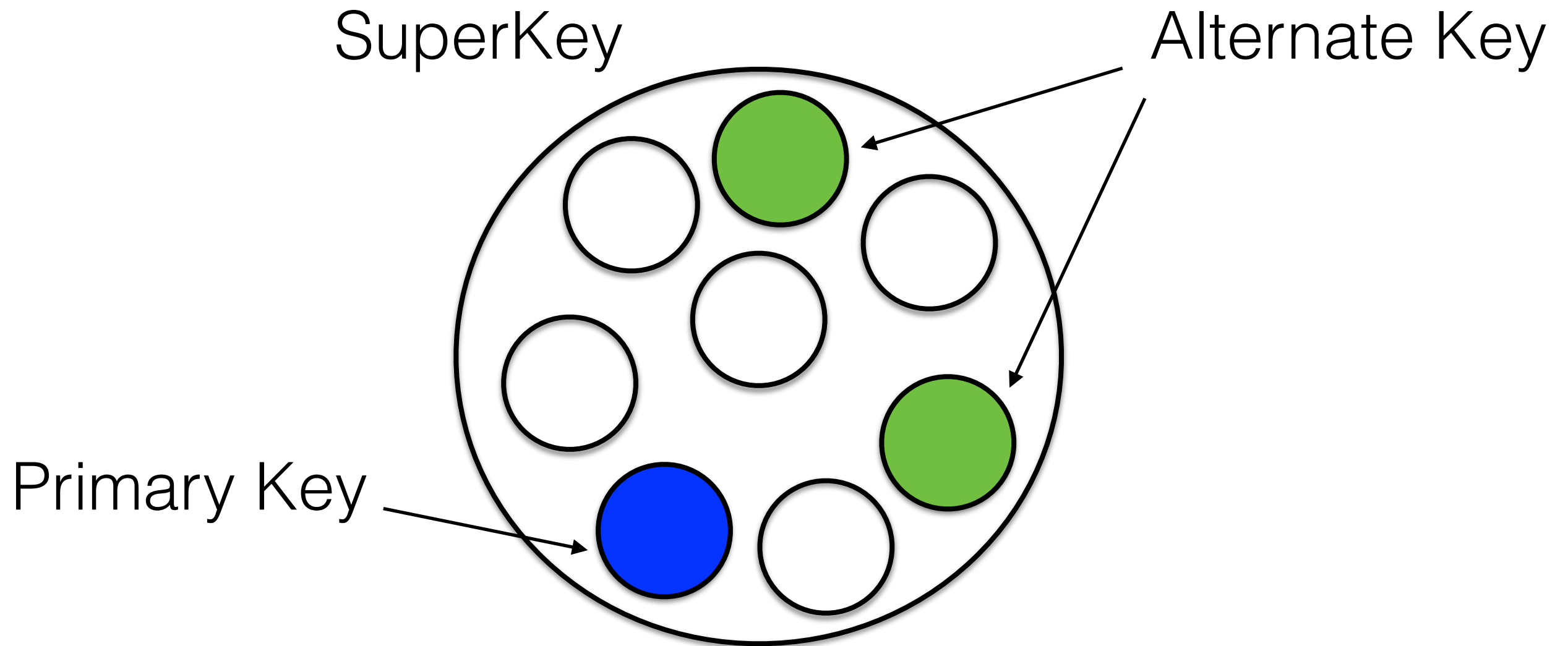
A Relation

Primary key

- * **Primary key** is the candidate key that is selected to identify tuples uniquely within the relation.
- * A relation always has a primary key.
- * The candidate keys that are not selected to be the primary key are called **alternate keys**.
- * Sometimes row-ids or sequential numbers are assigned as keys to identify the rows in a table called **surrogate key**.

Primary Key and Alternate Key

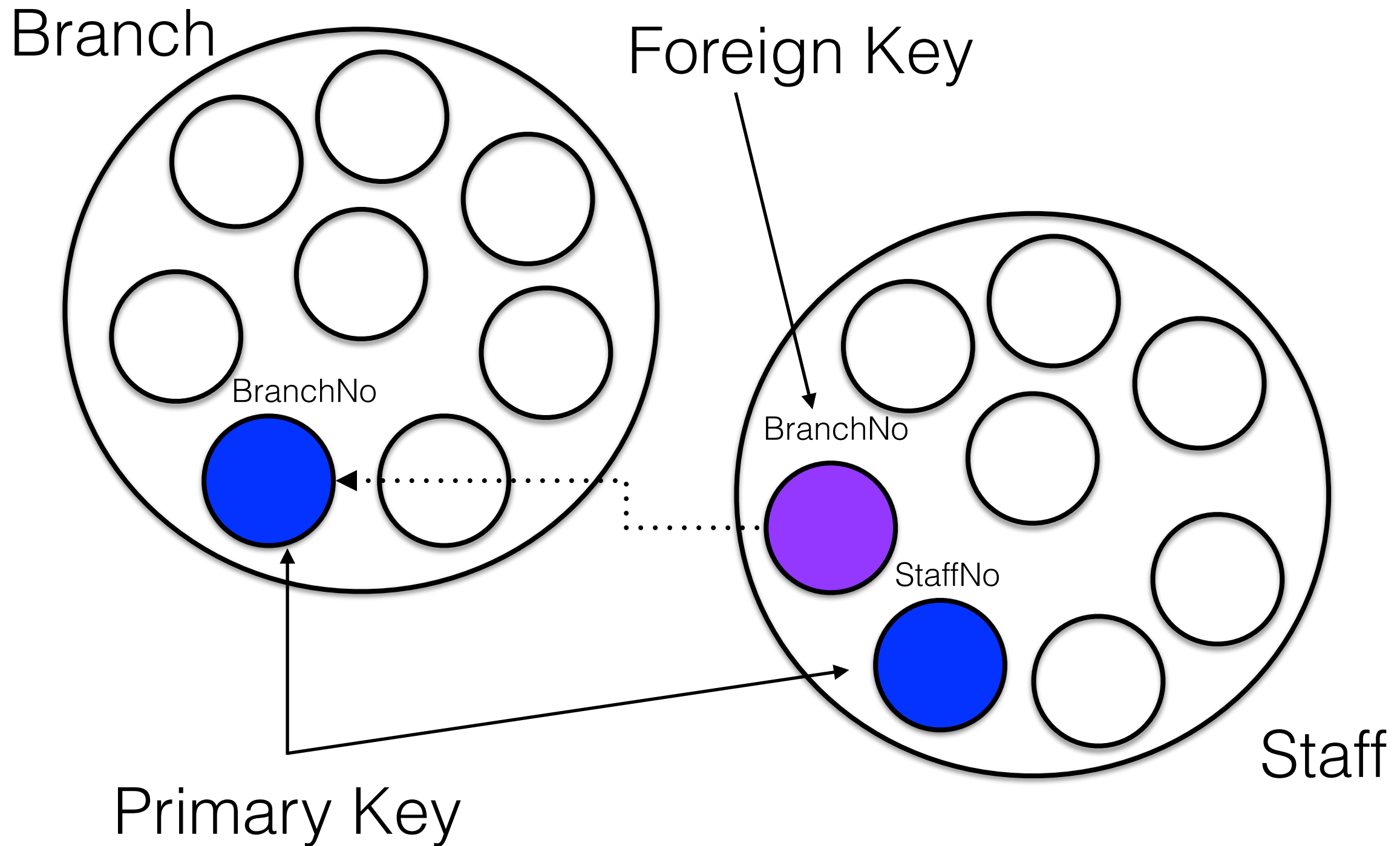
Students (StudentNo, Firstname, Lastname, DOB, Email)



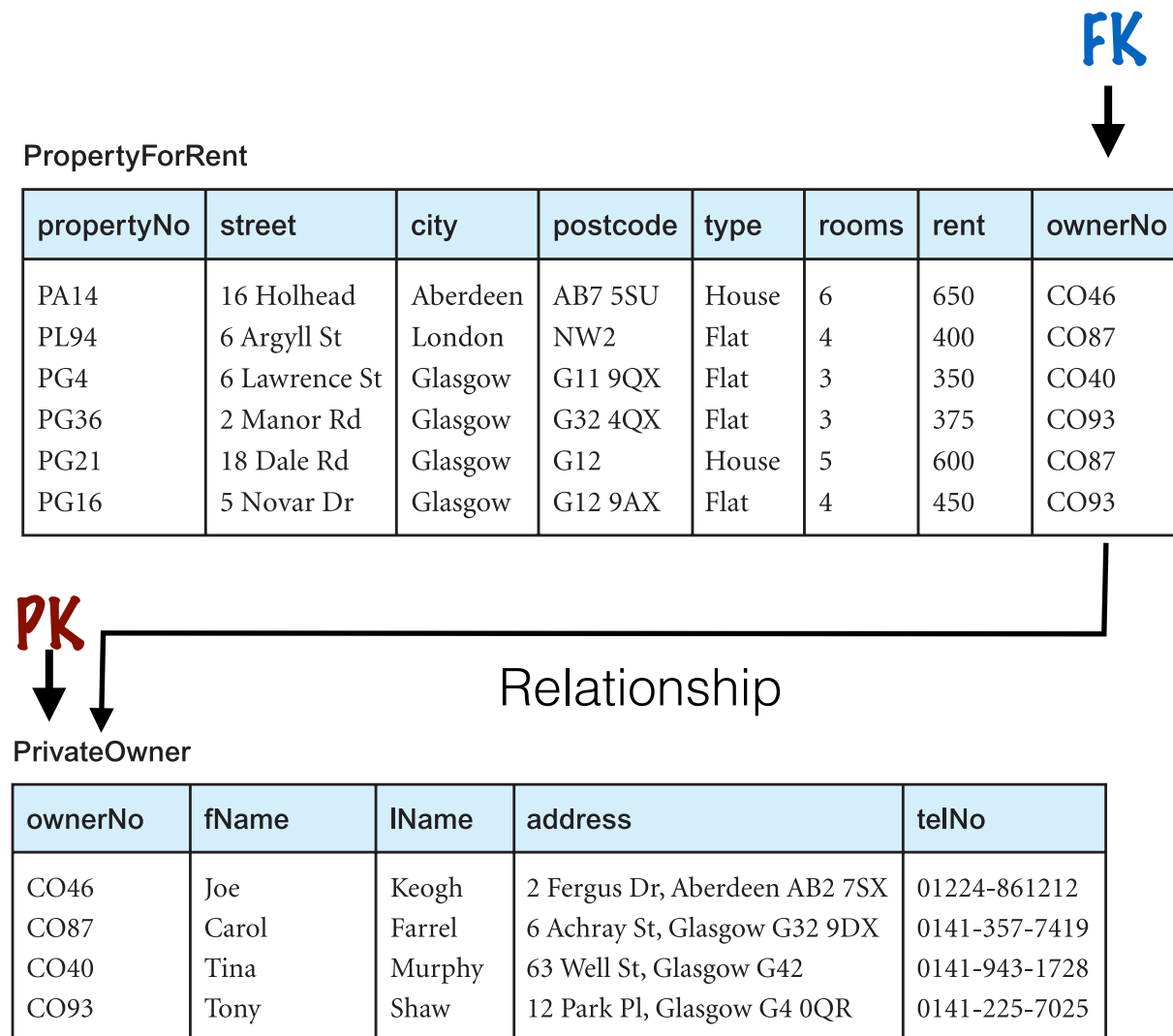
Foreign key

- * Foreign key is an attribute, or set of attributes, within one relation that matches the candidate key of some (possibly the same) relation.
- * When an attribute appears in more than one relation, its appearance usually **represents a relationship between tuples of the two relations.**

Primary Key and Foreign Key



Primary Key and Foreign Key



- The value in the foreign key column (or columns) FK of the referencing relation R1 can be either:
 - (1) a value of an existing primary key value of a corresponding primary key PK in the referenced relation R2, or
 - (2) a null.

Integrity constraint

- ***Referential integrity Constraint**

- ***FK-Referential Actions:** When an UPDATE or DELETE operation affects a key value in the parent table that has matching rows in the child table, the result depends on the referential action specified by ON UPDATE and ON DELETE subclauses of the FOREIGN KEY clause.

- ***RESTRICT:** Rejects the delete or update operation for the parent table. Specifying RESTRICT (or **NO ACTION**) is the same as omitting the ON DELETE or ON UPDATE clause.

Integrity constraint

*FK-Referential Actions:

***CASCADE**: Delete or update the row from the parent table and automatically delete or update the matching rows in the child table. Both **ON DELETE CASCADE** and **ON UPDATE CASCADE** are supported. Between two tables, do not define several **ON UPDATE CASCADE** clauses that act on the same column in the parent table or in the child table.

***SET NULL**: Delete or update the row from the parent table and set the foreign key column or columns in the child table to **NULL**. Both **ON DELETE SET NULL** and **ON UPDATE SET NULL** clauses are supported. If you specify a **SET NULL** action, make sure that you have not declared the columns in the child table as **NOT NULL**.

Example#1

EMP
PK

EMPNO	ENAME	DEPTNO
100	A	10
101	B	20
102	C	20
103	D	30

FK



DEPT
PK

DEPTNO	DNAME
10	IT
20	HR
30	Sales
40	Marketing

1. **INSERT INTO emp VALUES (104, 'D', 50); ?**
2. **DELETE FROM dept WHERE deptno = 20 ; ?**

Example#2

EMP
PK

FK; ON DELETE CASCADE

DEPT
PK

EMPNO	ENAME	DEPTNO
100	A	10
101	B	20
102	C	20
103	D	30

DEPTNO	DNAME
10	IT
20	HR
30	Sales
40	Marketing

1. **INSERT INTO emp VALUES (104, 'D', 50); ?**
2. **DELETE FROM dept WHERE deptno = 20 ; ?**

Example#3

EMP
PK

FK; ON DELETE SET NULL

DEPT
PK

EMPNO	ENAME	DEPTNO
100	A	10
101	B	20
102	C	20
103	D	30

DEPTNO	DNAME
10	IT
20	HR
30	Sales
40	Marketing

1. **INSERT INTO emp VALUES (104, 'D', 50); ?**
2. **DELETE FROM dept WHERE deptno = 20 ; ?**

Integrity constraint

- * **General constraints** (Unique, Check, Not Null)

- * Additional rules specified by the users or database administrators of a database that define or constrain some aspect of the enterprise.

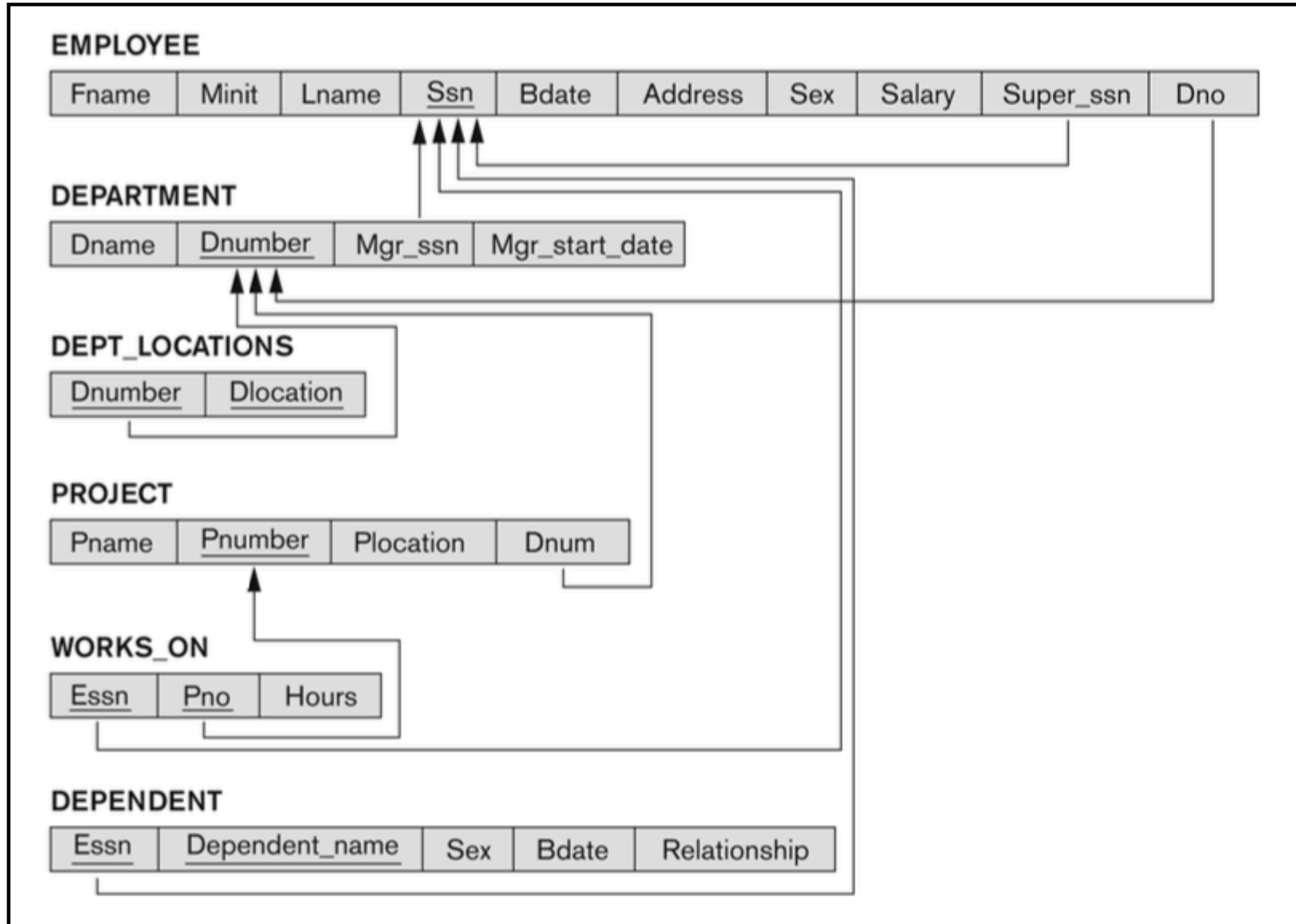
- * Null: represents a value for an attribute that is currently unknown or is not applicable (N/A) or no value has yet been supplied for this tuple.

- * Example: **Check** (salary > 15000)

Example: COMPANY database

Q1: How many foreign keys are there in each table?

Q2: Please specify the order of table creation.



Possible Violations for Each Operation

***INSERT** may violate any of the constraints:

- *Domain constraints**

- * if one of the attribute values provide for the new tuple is not of the specified attribute domain.

- *Key constraints**

- * if value of a key attribute in the new tuple already exists in another tuple in the relation.

- *Referential integrity constraints**

- * if a foreign key value in the new tuple references a primary key value that does not exist in the referenced relation.

- *Entity integrity constraints**

- * if the primary key value is null in the new tuple.

Possible Violations for Each Operation

- ***DELETE** may violate only referential integrity:

- *Referential integrity constraints

- * if the primary key value of the tuple being deleted is referenced from other tuples in the database.

- *Can be remedied by several actions:

- ***RESTRICT** option: reject the deletion

- ***CASCADE** option: propagate the new primary key value into the foreign keys of the referencing tuples

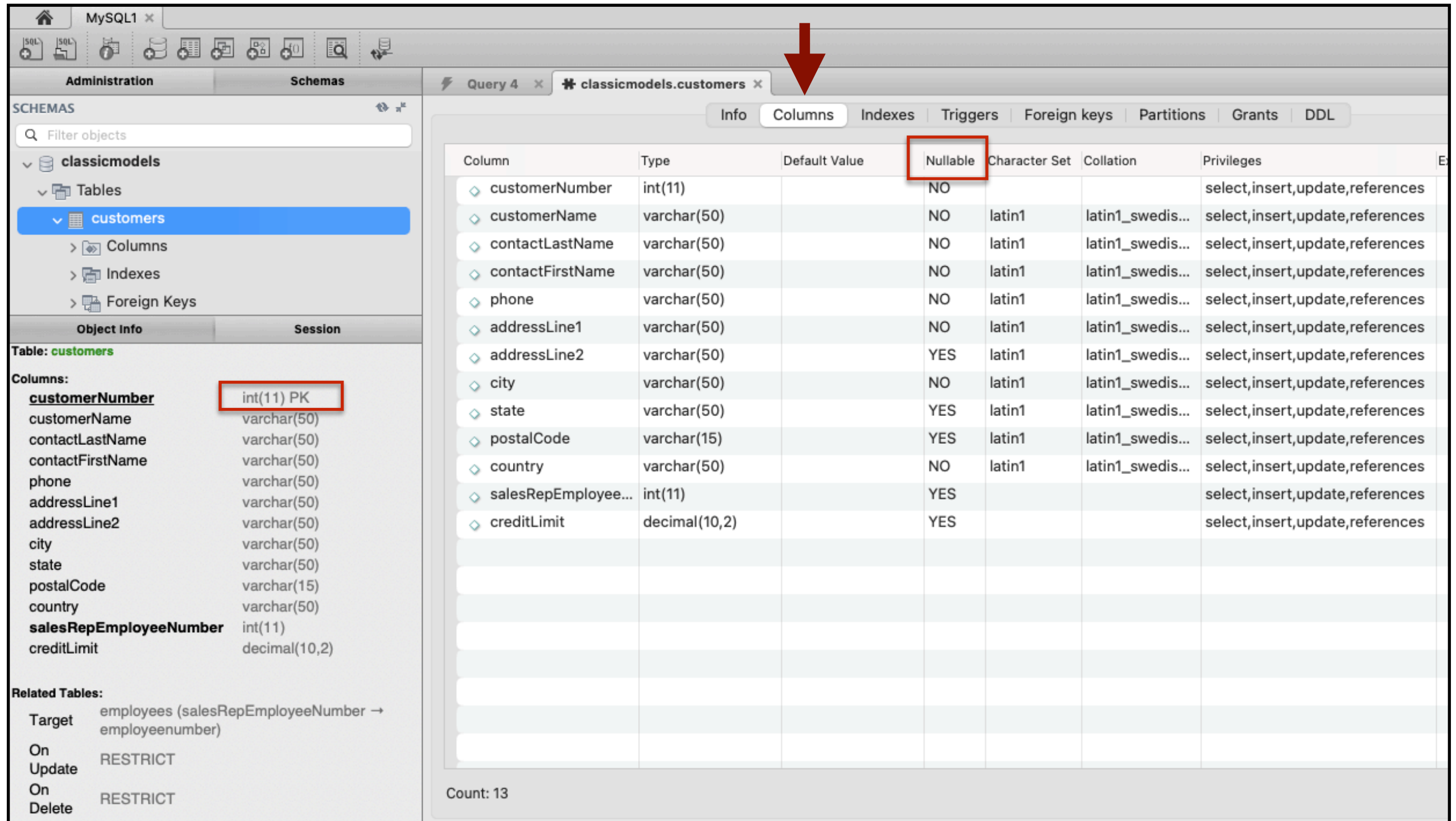
- ***SET NULL** option: set the foreign keys of the referencing tuples to NULL.

- *One of the above options must be specified during database design for each foreign key constraint.

Possible Violations for Each Operation

- ***UPDATE** may violate domain constraint and NOT NULL constraint on an attribute being modified:
- *Any of the other constraints may also be violated, depending on the attribute being updated:
 - *Updating the primary key
 - *Updating a foreign key

MySQL Workbench



The screenshot displays the MySQL Workbench interface with the 'classicmodels.customers' table selected. The 'Columns' tab is active, showing a list of columns with their types, default values, and nullability. A red arrow points to the 'Columns' tab, and a red box highlights the 'Nullable' column in the table structure grid.

Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
customerNumber	int(11)		NO			select,insert,update,references
customerName	varchar(50)		NO	latin1	latin1_swedis...	select,insert,update,references
contactLastName	varchar(50)		NO	latin1	latin1_swedis...	select,insert,update,references
contactFirstName	varchar(50)		NO	latin1	latin1_swedis...	select,insert,update,references
phone	varchar(50)		NO	latin1	latin1_swedis...	select,insert,update,references
addressLine1	varchar(50)		NO	latin1	latin1_swedis...	select,insert,update,references
addressLine2	varchar(50)		YES	latin1	latin1_swedis...	select,insert,update,references
city	varchar(50)		NO	latin1	latin1_swedis...	select,insert,update,references
state	varchar(50)		YES	latin1	latin1_swedis...	select,insert,update,references
postalCode	varchar(15)		YES	latin1	latin1_swedis...	select,insert,update,references
country	varchar(50)		NO	latin1	latin1_swedis...	select,insert,update,references
salesRepEmployee...	int(11)		YES			select,insert,update,references
creditLimit	decimal(10,2)		YES			select,insert,update,references

Count: 13

MySQL Workbench

The screenshot shows the MySQL Workbench interface with the 'classicmodels' database selected. The 'Foreign keys' tab is active, displaying a list of foreign keys. A red arrow points to the 'Foreign keys' tab, and a black arrow points from the 'salesRepEmployeeNumber' column in the 'customers' table to the 'salesRepEmployeeNumber' column in the 'employees' table in the foreign key list.

Table: customers

Columns:

Column	Type
customerNumber	int(11) PK
customerName	varchar(50)
contactLastName	varchar(50)
contactFirstName	varchar(50)
phone	varchar(50)
addressLine1	varchar(50)
addressLine2	varchar(50)
city	varchar(50)
state	varchar(50)
postalCode	varchar(15)
country	varchar(50)
salesRepEmployeeNumber	int(11)
creditLimit	decimal(10,2)

Related Tables:

Target	Relationship
employees	(salesRepEmployeeNumber → employeenumber)

Foreign Keys:

Name	Schema	Table	Column	Referenced Sch...	Referenced Table	Referenced Colu...
customers_ibfk_1	classicmodels	customers	salesRepEmpl...	classicmodels	employees	employeenum...
orders_ibfk_1	classicmodels	orders	customerNum...	classicmodels	customers	customernum...
payments_ibfk_1	classicmodels	payments	customerNum...	classicmodels	customers	customernum...

Count: 3